

11. Reference Type

Ye advanced topic hai, sirf edge cases samajhne ke liye. Na bhi padho to chalega.

Problem

Kabhi kabhi method call karte waqt `this` kho jaata hai:

```
js

let user = {
  name: "John",
  hi() { console.log(this.name); },
  bye() { console.log("Bye"); }
};

user.hi(); // kaam karta hai

(user.name == "John" ? user.hi : user.bye)(); // Error! this = undefined
```

Kyun?

`obj.method()` me asal me **do kaam** hote hain:

1. Dot `.` property `obj.method` nikalta hai.
2. Bracket `()` usse chalata hai.

Agar dono ko alag kar do, `this` gayab:

```
js

let hi = user.hi;
hi(); // Error, this undefined
```

Reference Type kya hai?

- JS andar ek **special internal type** use karta hai: *Reference Type*.
- Dot `.` function nahi, ek triple `(base, name, strict)` return karta hai.
 - `base` = object, `name` = property ka naam, `strict` = strict mode hai ya nahi.
 - `user.hi` ka reference: `(user, "hi", true)`
- Jab `()` is reference par lagta hai, use pura pata hota hai ki object kaun hai, to sahi `this` set ho jaata hai.
- Lekin **koi bhi aur operation** (assignment `=`, `||`, conditional `? :`) reference ko hata kar seedha value (function) bana deta hai. Fir `this` nahi milta.

Kab `this` sahi milta hai?

Sirf jab call seedha `obj.method()` ya `obj['method']()` ho.

Fix

`func.bind()` jaise tarike use karo.

Tasks ke tricks

```
js

let user = {
  name: "John",
  go: function() { alert(this.name) }
}           // <- yaha semicolon nahi hai!

(user.go)() // Error
```

Semicolon na hone se JS isse `{...}(user.go)()` samajhta hai. Semicolon lagao to theek.

`(user.go)()` me brackets kuch nahi badalte.

```
js

obj.go();           // (1) sahi this
(obj.go)();         // (2) sahi this
(method = obj.go)(); // (3) undefined (assignment reference hata deta hai)
(obj.go || obj.stop)(); // (4) undefined (|| bhi hata deta hai)
```

Yaad rakho

Property nikalne `(.)` aur call karne `(( ))` ke beech koi aur operation aaya to `this` chala jaata hai.